

CTE

CTE

- CTE stands for common table expression
- Temporary, named result set (virtual table), that can be used multiple times within your query to simplify and organize complex query
- Create a temporary result, give it a name, then use that name in the next query
- Basics syntax

```
WITH cte_name AS (  
    SELECT  
        column1,  
        column2  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name;
```

Why use CTEs?

- The main reason we use a CTE is to make complex SQL queries easier to write, read, and maintain.
- Avoiding a complicated nested subquery
- Break complex query into steps
- CTEs are very useful with window functions

■ Let's give the intermediate query result a name first, then use it.

```
USE MyDatabase;
```

```
-- egypt customers
```

```
WITH egypt_customers AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score  
    FROM customers  
    WHERE country = 'Egypt'  
)  
SELECT *  
FROM egypt_customers;
```

```
-- High-score customers
```

```
WITH high_score_customers AS (  
    SELECT  
        id,  
        first_name,
```

```

        country,
        score
    FROM customers
    WHERE score >= 85
)
SELECT *
FROM high_score_customers;

-- Calculate the average score for each country:
WITH country_scores AS (
    SELECT
        country,
        AVG(score) AS avg_score
    FROM customers
    GROUP BY country
)
SELECT *
FROM country_scores;

-- Find countries where the average score is greater than 80:
WITH country_scores AS (
    SELECT
        country,
        AVG(score) AS avg_score
    FROM customers
    GROUP BY country
)
SELECT
    country,
    avg_score
FROM country_scores
WHERE avg_score > 80;

-- Find the average score of Egyptian customers, then find Egyptian customers above that
average:
WITH egypt_customers AS (
    SELECT
        id,
        first_name,
        score
    FROM customers
    WHERE country = 'Egypt'
),
egypt_average AS (
    SELECT
        AVG(score) AS avg_score
    FROM egypt_customers
)
SELECT
    e.id,
    e.first_name,
    e.score
FROM egypt_customers e
CROSS JOIN egypt_average a
WHERE e.score > a.avg_score;

```

-- Find the top 3 customers from each country:

```
WITH ranked_customers AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score,  
        ROW_NUMBER() OVER (  
            PARTITION BY country  
            ORDER BY score DESC  
        ) AS rn  
    FROM customers  
)  
SELECT  
    id,  
    first_name,  
    country,  
    score,  
    rn  
FROM ranked_customers  
WHERE rn <= 3;
```

-- If customers have the same score and you want them to have the same ranking:

```
WITH ranked_customers AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score,  
        RANK() OVER (  
            PARTITION BY country  
            ORDER BY score DESC  
        ) AS customer_rank  
    FROM customers  
)  
SELECT *  
FROM ranked_customers  
WHERE customer_rank <= 3;
```

-- Create score categories:

```
WITH customer_levels AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score,  
        CASE  
            WHEN score >= 90 THEN 'Excellent'  
            WHEN score >= 80 THEN 'Good'  
            WHEN score >= 70 THEN 'Average'  
            ELSE 'Poor'  
        END AS score_level  
    FROM customers  
)  
SELECT *  
FROM customer_levels;
```

```
-- Clean the names → select high-score customers → rank them by country.
```

```
WITH cleaned_customers AS (  
    SELECT  
        id,  
        TRIM(first_name) AS first_name,  
        country,  
        score  
    FROM customers  
) ,
```

```
high_score_customers AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score  
    FROM cleaned_customers  
    WHERE score >= 80  
) ,
```

```
ranked_customers AS (  
    SELECT  
        id,  
        first_name,  
        country,  
        score,  
        ROW_NUMBER() OVER (  
            PARTITION BY country  
            ORDER BY score DESC  
        ) AS rn  
    FROM high_score_customers  
)
```

```
SELECT  
    id,  
    first_name,  
    country,  
    score,  
    rn  
FROM ranked_customers  
WHERE rn <= 3;
```

```
-- Count customers by country
```

```
WITH country_count AS (  
    SELECT  
        country,  
        COUNT(*) AS total_customers  
    FROM customers  
    GROUP BY country  
)  
SELECT *  
FROM country_count  
ORDER BY total_customers DESC;
```

```
-- Countries with more than 5 customers
```

```
WITH country_count AS (  
    SELECT  
        country,
```

```

        COUNT(*) AS total_customers
    FROM customers
    GROUP BY country
)
SELECT *
FROM country_count
WHERE total_customers > 5;

-- Maximum score by country
WITH country_max_score AS (
    SELECT
        country,
        MAX(score) AS max_score
    FROM customers
    GROUP BY country
)
SELECT *
FROM country_max_score;

-- Minimum score by country
WITH country_min_score AS (
    SELECT
        country,
        MIN(score) AS min_score
    FROM customers
    GROUP BY country
)
SELECT *
FROM country_min_score;

-- Highest-scoring customer
WITH max_score AS (
    SELECT MAX(score) AS maximum_score
    FROM customers
)
SELECT
    c.id,
    c.first_name,
    c.country,
    c.score
FROM customers c
CROSS JOIN max_score m
WHERE c.score = m.maximum_score;

-- Second-highest score
WITH ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        DENSE_RANK() OVER (
            ORDER BY score DESC
        ) AS ranking
    FROM customers
)

```

```

SELECT *
FROM ranked_customers
WHERE ranking = 2;

-- Top 3 customers overall
WITH ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        ROW_NUMBER() OVER (
            ORDER BY score DESC
        ) AS rn
    FROM customers
)
SELECT *
FROM ranked_customers
WHERE rn <= 3;

-- Top 3 customers from each country
WITH ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        ROW_NUMBER() OVER (
            PARTITION BY country
            ORDER BY score DESC
        ) AS rn
    FROM customers
)
SELECT *
FROM ranked_customers
WHERE rn <= 3;

-- Rank every customer
WITH ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        RANK() OVER (
            ORDER BY score DESC
        ) AS ranking
    FROM customers
)
SELECT *
FROM ranked_customers
ORDER BY ranking;

-- Rank customers within their country
WITH ranked_customers AS (
    SELECT
        id,

```

```

        first_name,
        country,
        score,
        RANK() OVER (
            PARTITION BY country
            ORDER BY score DESC
        ) AS country_rank
    FROM customers
)
SELECT *
FROM ranked_customers
ORDER BY country, country_rank;

-- Customers above their country's average
WITH country_average AS (
    SELECT
        country,
        AVG(score) AS avg_score
    FROM customers
    GROUP BY country
)
SELECT
    c.id,
    c.first_name,
    c.country,
    c.score,
    ca.avg_score
FROM customers c
JOIN country_average ca
    ON c.country = ca.country
WHERE c.score > ca.avg_score;

-- Add a score category
WITH customer_categories AS (
    SELECT
        id,
        first_name,
        country,
        score,
        CASE
            WHEN score >= 90 THEN 'Excellent'
            WHEN score >= 80 THEN 'Good'
            WHEN score >= 70 THEN 'Average'
            ELSE 'Poor'
        END AS category
    FROM customers
)
SELECT *
FROM customer_categories;

-- Count customers in each score category
WITH customer_categories AS (
    SELECT
        id,
        first_name,
        country,
        score,
        CASE

```

```

        WHEN score >= 90 THEN 'Excellent'
        WHEN score >= 80 THEN 'Good'
        WHEN score >= 70 THEN 'Average'
        ELSE 'Poor'
    END AS category
FROM customers
),

category_count AS (
    SELECT
        category,
        COUNT(*) AS total_customers
    FROM customer_categories
    GROUP BY category
)

SELECT *
FROM category_count;

-- Clean + filter + rank
WITH cleaned_customers AS (
    SELECT
        id,
        TRIM(first_name) AS first_name,
        country,
        score
    FROM customers
),

high_score AS (
    SELECT *
    FROM cleaned_customers
    WHERE score >= 80
),

ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        ROW_NUMBER() OVER (
            PARTITION BY country
            ORDER BY score DESC
        ) AS rn
    FROM high_score
)

SELECT *
FROM ranked_customers
WHERE rn <= 3;

-- Find countries with the highest average score
WITH country_average AS (
    SELECT
        country,
        AVG(score) AS avg_score
    FROM customers

```



```

        GROUP BY country
    ),

ranked_countries AS (
    SELECT
        country,
        avg_score,
        RANK() OVER (
            ORDER BY avg_score DESC
        ) AS ranking
    FROM country_average
)

SELECT *
FROM ranked_countries
WHERE ranking = 1;

-- Find the highest-scoring customer per country
WITH ranked_customers AS (
    SELECT
        id,
        first_name,
        country,
        score,
        ROW_NUMBER() OVER (
            PARTITION BY country
            ORDER BY score DESC
        ) AS rn
    FROM customers
)
SELECT
    id,
    first_name,
    country,
    score
FROM ranked_customers
WHERE rn = 1;

```